



Список тренингов / Промпт-инжиниринг / Промпт-инжиниринг 2.0 / Модуль 10. Кейсы
(/teach/control/stream/view/id/931478155)

PErr10.1. Генератор логотипов с использованием Yandex-ART API. Часть 1 (new)

Следующий урок (/pl/teach/control/lesson/view?
id=346712532)

Дорогой студент! Интерфейсы современных приложений могут очень быстро меняться. Если вдруг то, что видишь ты, отличается от того, что демонстрирует эксперт, настолько, что тебе непонятно, куда нажимать — напиши об этом в учебный чат, указав номер урока и видео. Наш куратор обязательно поможет тебе разобраться, а мы постараемся как можно скорее актуализировать материал :)

Что мы умеем

- ✓ Работаем с Yandex Art через API для генерации изображений.
- ✓ Настраиваем сервисные аккаунты, API-ключи и .env-файлы в Yandex Cloud.
- ✓ Создаем Python-код для запросов к API, сохранения изображений и тестирования генерации.

Результат дня

Действия

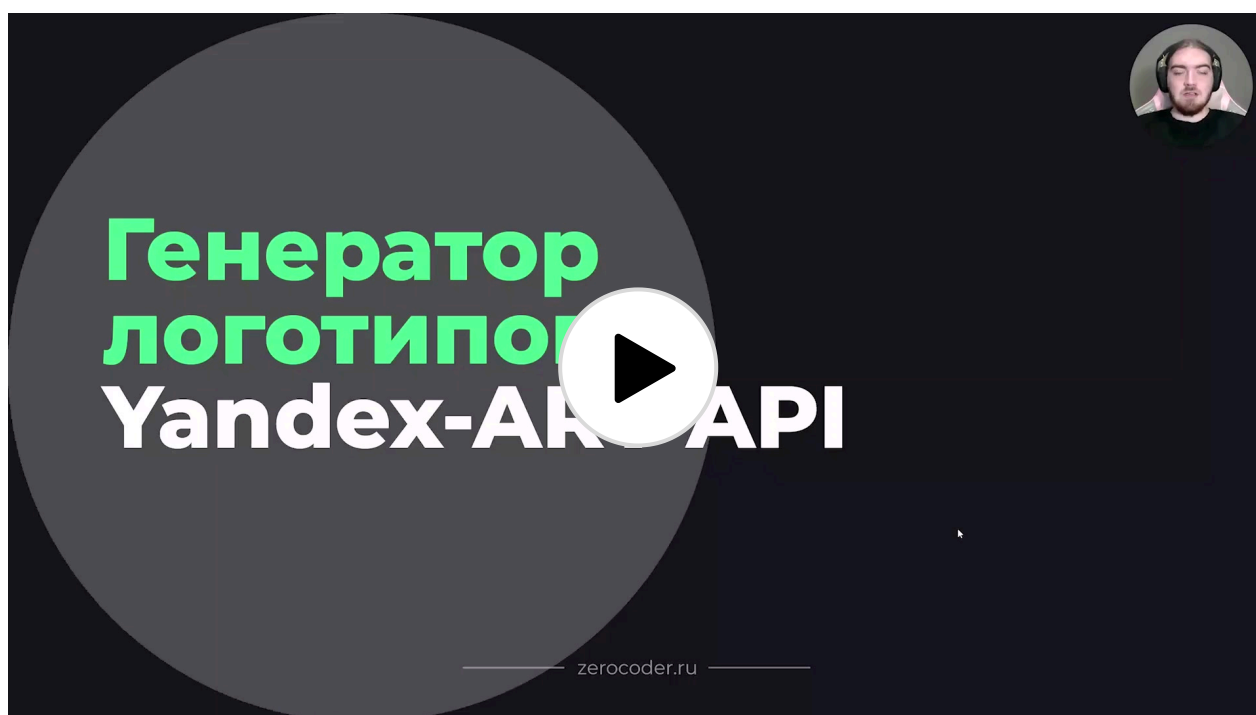
Узнаешь, как работать с Yandex-ART через API для создания веб-приложения, которое будет генерировать логотипы.

▶ Время чтения и просмотра: ~ 1,5 часа

🕒 Время выполнения задания: ~ 45 минут

Теория на сегодня

1. Цели урока



Сегодня мы создадим небольшое приложение, которое делает запрос к Yandex для генерации изображений и сохраняет полученные результаты. Наша финальная задача — создание генератора логотипов.

Урок ведёт Даниил Соболев, практикующий Python-разработчик, ежедневно интегрирующий Yandex, OpenAI и другие нейросети в реальные проекты. Это позволяет ему оценивать учебные проекты, в том числе сегодняшний, с точки зрения бизнес-применимости, востребованности в различных сферах и реалистичности подобных решений на сегодняшний день.

Действия

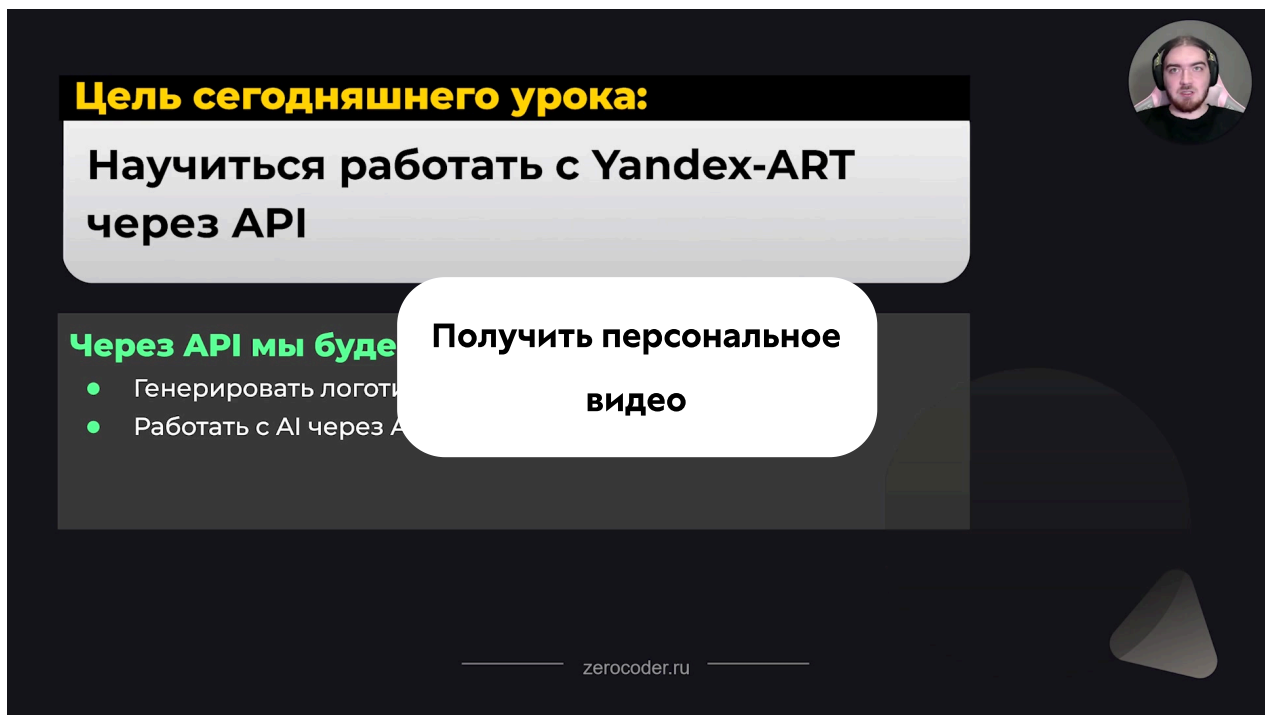
Сегодня мы:

- Разберёмся, что можно делать с помощью Yandex Art и его API, изучим необходимые инструменты;
- Обсудим, что такое API, как оно устроено и работает. Это понимание упростит дальнейшую работу и интеграции;
- Реализуем функциональные блоки (методы и функции) для отправки запросов, получения ответов и сохранения сгенерированных изображений на локальном компьютере;

Эта система станет основой для генератора логотипов. В следующем уроке мы обернём эту функциональность во Flask-фреймворк для создания веб-сайта с пользовательским интерфейсом. Это превратит наше консольное приложение в готовый продукт, удобный для конечного пользователя, который предпочитает стандартный формат взаимодействия через сайт работе с кодом.

Итоговая цель урока: научиться работать с Yandex Art через API, включая взаимодействие с платформой Yandex Cloud, и понять общие принципы работы с API, которые пригодятся для интеграции с любыми подобными сервисами.

API, методы запроса и другие инструменты, с которыми мы будем взаимодействовать — не уникальны исключительно для Yandex. Всё то, что мы будем использовать сегодня, можно использовать и для GigaChat, OpenAI, Perplexity, DeepSeek и т.д. Все эти нейросети предоставляют возможность запроса к ним через API.



▼ Возможности Yandex Art

Рассмотрим ключевые возможности Yandex Art.

1. **Преобразование текста в изображение:** стандартная функция, где модель по промпту (текстовому описанию) создаёт изображение.
2. **Гибкость стилей:** модель должна поддерживать множество стилей, что критично для генератора логотипов, где запросы клиентов могут быть разнообразными.
3. **Доступ через API:** возможность программного взаимодействия с моделью необходима для создания собственного интерфейса (например, сайта), через который пользователь будет работать с Yandex Art.

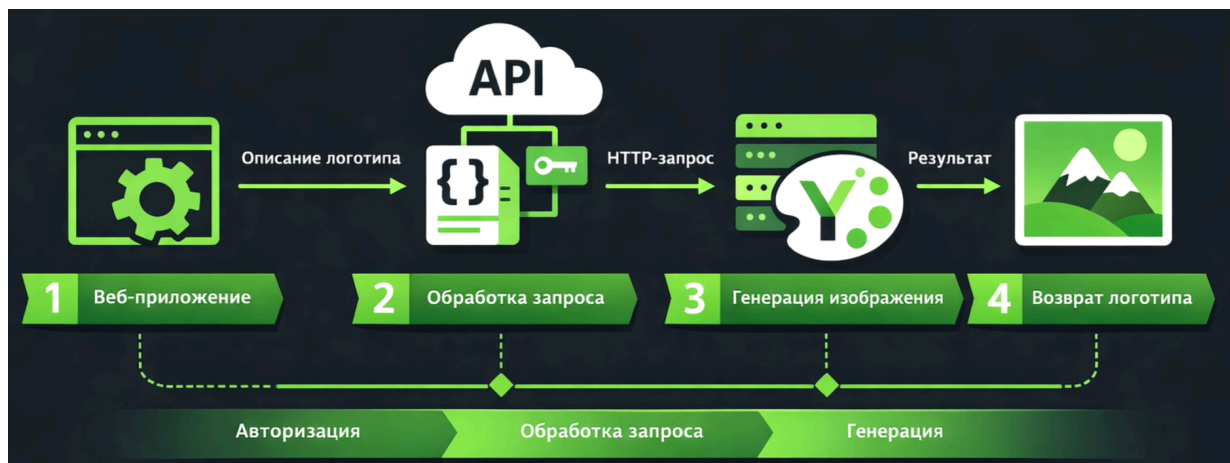
▼ Что такое API и почему мы его используем

- ✓ **API (Application Programming Interface)** — это набор правил и инструментов для создания запросов.

В нашем сценарии:

1. Код на нашем компьютере формирует запрос и отправляет его через интернет на сервер Yandex.
2. Сервер Yandex принимает запрос и обрабатывает его.
3. Сервер генерирует изображение.
4. И отправляет назад ответ (в нашем случае — логотип).

Действия



В большинстве случаев мы используем API для получения данных, которых нет локально. Типичные примеры:

- Боту нужны актуальные данные о погоде в разных регионах — он обращается к погодному API;
- Приложению требуется текущий курс валют — оно делает запрос к финансовому API.

Множество сервисов предоставляют такой доступ. Запросы в интернете обрабатываются за доли секунды, позволяя практически мгновенно получать актуальную информацию непосредственно с сервера, где эти данные хранятся и обновляются.

В нашем случае мы будем не получать информацию, а отправлять запрос, и сервер будет создавать то, что нам необходимо создать, и отправлять нам ответ.

▼ Сравнение подходов: API vs локальная модель

На самом деле для решения нашей задачи можно использовать несколько методов, и среди них можно выделить два основных:

1. Использование API;
2. Развёртывание собственной локальной модели.

Мы выбираем API по ряду критических причин.

Недостатки использования локальной модели:

1. Низкая производительность и высокие затраты на её обеспечение.

Локальные модели, как правило, работают медленнее. Ускорить их работу можно, но это требует огромных финансовых вложений в высокопроизводительное «железо» (GPU, большие объёмы памяти) для проведения вычислений.

Действия

2. **Сложность развёртывания и поддержки.** Нам пришлось бы самостоятельно заниматься всей инфраструктурой: настраивать, обновлять и поддерживать работу модели. Это требует глубоких экспертных знаний о внутреннем устройстве алгоритмов, процессах развёртывания и оптимизации, что делает подход чрезвычайно сложным и трудозатратным.

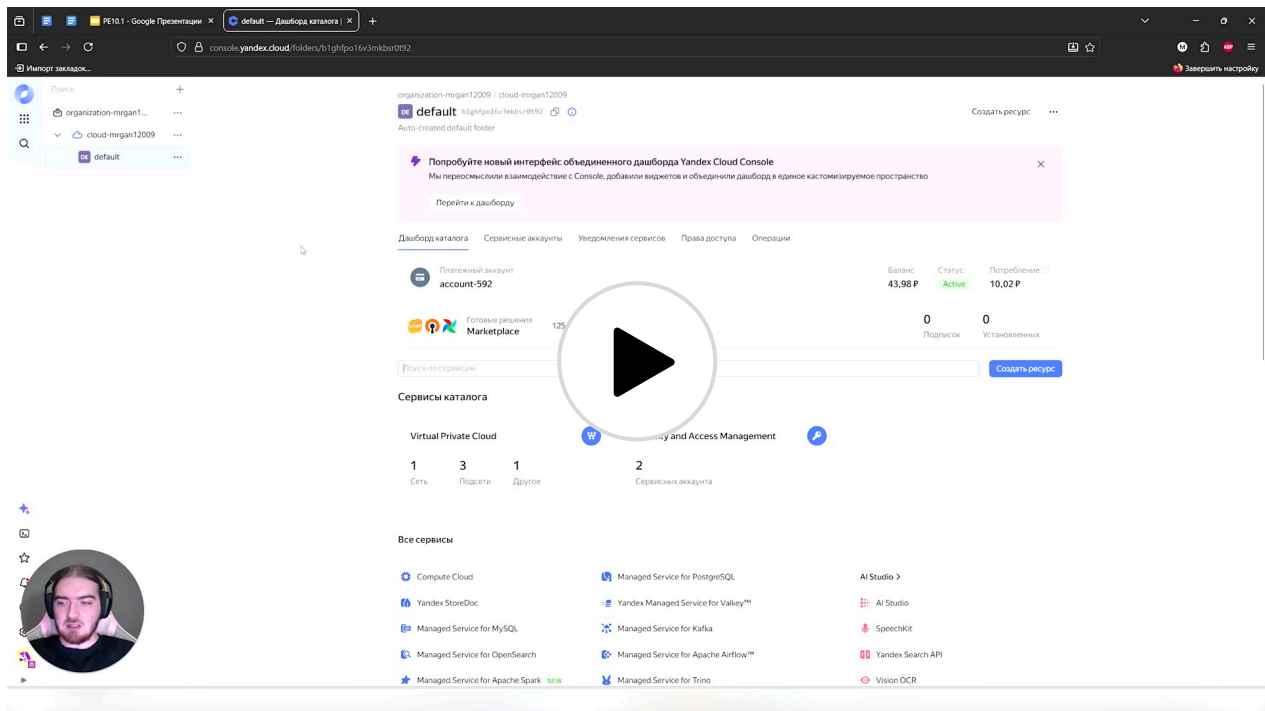
Преимущества облачного API:

1. **Экономическая эффективность и экономия ресурсов.** API избавляет нас от необходимости покупать, настраивать и обслуживать дорогостоящие серверы. Мы платим только за потребляемые ресурсы — в нашем случае за каждое сгенерированное изображение. Средняя стоимость запроса составляет порядка 15-20 рублей, что оптимально для учебного или небольшого коммерческого проекта.
2. **Масштабируемость и простота архитектуры.** Этот подход остаётся экономичным даже при размещении нашего приложения на арендованном сервере. Аренда стандартного сервера для запуска нашего кода обойдётся недорого (порядка 210 рублей в месяц). В то же время, аренда сервера, достаточно мощного для запуска локальной генеративной модели, будет стоить на несколько порядков дороже (несколько тысяч рублей в сутки). Наша архитектура, основанная на API, обеспечивает невысокую стоимость запросов и простоту реализации.
3. **Универсальность и переносимость.** Принципы работы с API едины для разных провайдеров: Yandex Art, OpenAI, GigaChat, Perplexity и другие. Запросы составляются по схожим правилам, что позволяет легко адаптировать полученные навыки для работы с другими сервисами. Таким образом, API становится нашим универсальным и удобным инструментом, позволяющим делегировать ресурсоёмкую задачу генерации изображений мощному облачному сервису, сохраняя контроль над логикой приложения и минимизируя затраты.

▼ План практической работы

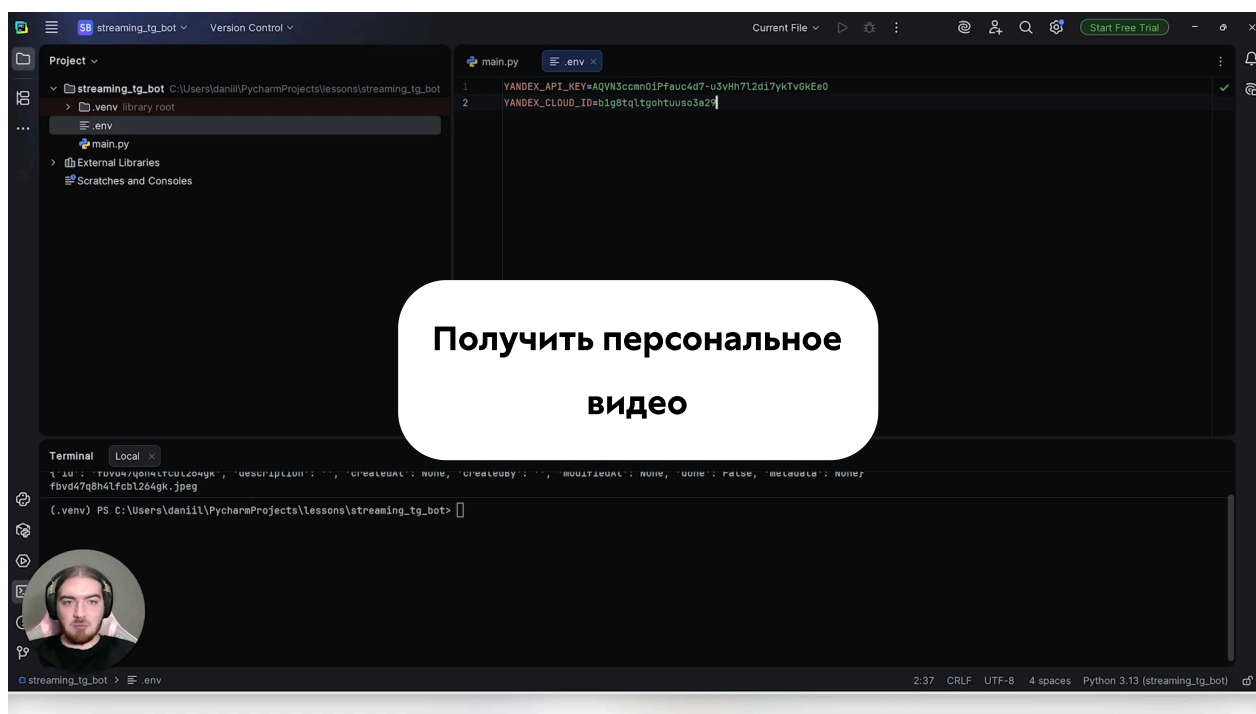
1. **Работа с Yandex Cloud:** мы зарегистрируемся на платформе, получим необходимые ключи доступа (API-ключи) для аутентификации наших запросов.
2. **Разработка кода:** на основе полученных ключей мы создадим функциональные блоки на Python для взаимодействия с API Yandex Art. Эта работа заложит фундамент для следующего урока, где мы создадим полноценное веб-приложение — генератор логотипов для клиентов.

2. Получение ключа в Yandex Cloud



► Регистрация и получение ключа в Yandex Cloud

3. Генерация изображений Yandex Art: секреты, запросы и тестирование



▼ Использование .env-файлов для защиты ключей

Обсудим важный момент — почему мы использовали файл `.env` и сохранили данные именно в нём.

✓ **Файл `.env` (сокращение от `environment`)** — это простой текстовый файл, используемый в программировании для хранения переменных окружения, таких как ключи API, пароли к базам данных, настройки сервера и другие конфиденциальные данные. Он хранит информацию в формате **КЛЮЧ=значение**, что позволяет отделить конфигурацию от исходного кода, обеспечивая безопасность и гибкость.

`.env` — это стандартный формат хранения секретных данных. Фактически это минимальный, но обязательный уровень безопасности для любого проекта. Во всех проектах, с которыми мы будем работать, файл `.env` будет присутствовать.

Что именно мы храним в `.env` и почему он считается «секретным»:

- API-ключи;
- токены;
- любые другие чувствительные данные, которые нельзя показывать.

В нашем случае это API-ключ Яндекса. Если кто-то получит этот ключ и начнёт использовать его для запросов, запросы будут выполняться корректно, ^{Действия} ответы будут приходить — **но оплачивать эти запросы**

будет владелец ключа — то есть не тот, кто делает запросы, а тот, чей ключ используется. Очевидно, нам это не нужно.

Это касается и любых других сервисов: например, когда мы будем работать с Telegram-ботами, мы будем получать специальный токен, и если такой токен утечёт в сеть, кто-то сможет делать рассылки от имени нашего бота, писать нашим пользователям, управлять ботом без нашего ведома, и мы при этом полностью потеряем контроль над происходящим.

То же самое касается платёжных систем, терминалов, API платёжных шлюзов — все секретные данные всегда хранятся в `.env`.

Почему это удобно и безопасно:

- `.env` практически никогда не утекает случайно;
- GitHub, Docker и другие облачные и CI/CD-системы автоматически предупреждают нас, если мы пытаемся загрузить `.env` в репозиторий. Более того, эти системы активно мешают загрузке `.env`, потому что понимают, что внутри находятся критически важные данные.

Поэтому создание `.env` — это не рекомендация, а стандартная практика, которую мы просто принимаем как базу и используем всегда.

▼ Импорты и логика работы

Переходим в PyCharm. Именно в PyCharm мы сегодня будем работать и писать код. Сначала посмотрим на стандартный вариант отправки запросов, а затем реализуем модель для формирования запросов на Python, которую будем использовать дальше на практике.

Наша задача — не просто запустить готовый код, а понять, как он устроен и почему здесь используется именно такой набор инструментов. Для начала прочитаем код сверху вниз и разберём, как он работает.

1. Первое, что мы используем, — стандартная библиотека `requests`:

```
import requests
```

Действия

`requests` — это библиотека для отправки HTTP-запросов в интернете. По сути, с её помощью мы отправляем запрос с нашего локального компьютера на сервер Яндекса и получаем от него ответ.

2. Также нам понадобится `base64` :

```
import base64
```

`base64` нужен для того, чтобы разобрать изображение, которое вернётся от Яндекса. Изображение приходит не сразу в привычном формате `.jpg` или `.png`, а в закодированном виде — в формате `base64`.

 Любой файл на самом деле — это набор символов: цифр, букв, кодировок. То же самое касается изображений. Если открыть, например, PNG-файл «внутри», мы увидим не картинку, а описание того, какой пиксель где находится и какого он цвета.

На самом нижнем уровне это вообще нули и единицы, а чуть выше — различные кодировки. Нам такой ответ в виде строки не нужен. Нам нужно получить **реальное изображение**, поэтому мы будем декодировать эту строку и собирать из неё JPEG-файл.

3. Следующий импорт — `time` :

```
import time
```

`time` понадобится для циклических запросов. Генерация изображения занимает время, поэтому мы не можем получить результат мгновенно. Вместо того чтобы делать запросы к Яндексу 10–15 раз в секунду, мы будем опрашивать сервер с паузой — например, раз в 5 секунд.

Именно для этого ниже мы прописываем цикл `while` с `time.sleep`. Он позволяет подождать, проверить статус генерации, и как только Яндекс вернёт статус «Готово», забрать изображение и сохранить

Действия

его.

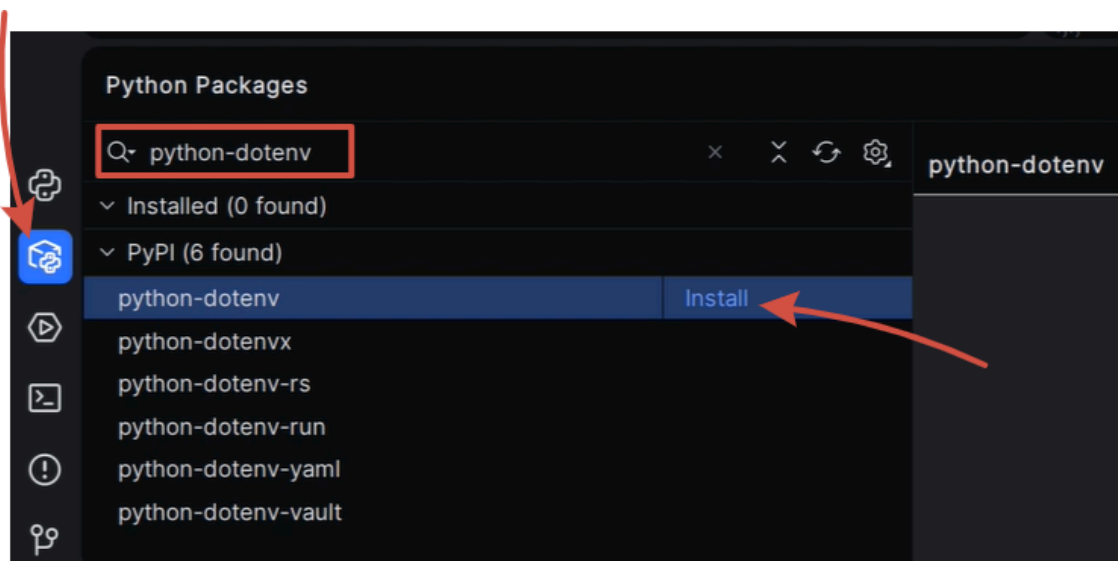
4. Также мы подключаем работу с переменными окружения:

```
from dotenv import load_dotenv
import os
```

Здесь мы используем библиотеку `python_dotenv`. Она нужна для того, чтобы загрузить переменные из файла `.env` — то есть из нашего переменного окружения.

5. Если библиотека ещё не установлена, её можно поставить стандартным способом: прописываем в терминале команду `pip install python_dotenv` и ждём завершения установки.

💡 Если по какой-то причине не получается установить через терминал, в PyCharm можно зайти в Python Packages, прописать в строке поиска «python-dotenv» и установить библиотеку оттуда — результат будет тот же самый.



6. Затем загружаем переменные окружения:

```
load_dotenv()
```

В момент выполнения этой строки Python:

- читает файл `.env` ;
 - загружает все переменные из этого файла;
- Действия

- делает их доступными через `os`.

7. После этого переходим к основной функции, с помощью которой будем отправлять запросы:

```
def yandex_art_request(prompt, seed):
```

Эта функция принимает два параметра:

- `prompt` — текст, который мы отправляем Яндексу для генерации изображения;
- `seed` — значение, которое влияет на воспроизводимость результата (если он задан).

▼ Структура запроса к Yandex Art: промпт, параметры и авторизация

Теперь разберём структуру самого запроса к Яндекс.Арт. Пойдём сверху вниз и посмотрим, из каких частей он состоит.

1. Первое, что нужно указать в запросе, — это Model URI. Это адрес модели, которую мы хотим использовать для генерации.

```
prompt = {  
    "modelUri": f"art://{os.getenv('YANDEX_CLOUD_ID')}/yandex-art/latest"
```

Разберём эту строку подробнее:

- `art://` — указывает, что мы используем сервис Yandex Art для генерации изображений;
- `os.getenv('YANDEX_CLOUD_ID')` — здесь мы подставляем ID нашего проекта в Яндекс.Облаке, который берётся из файла `.env`;
- `yandex-art/latest` — указывает, что мы используем модель Яндекс.Арт в самой последней версии.



Обрати внимание на форматирование строки: мы используем f-строку с фигурными скобками. Это значит, что в момент выполнения кода Python подставит значение переменной `YANDEX_CLOUD_ID` из файла `.env`, и строка примет окончательный вид с нашим реальным ID проекта.

Действия

Почему мы не пишем ID прямо в коде? Ответ простой: если мы оставим реальный ID или ключ прямо в коде, а затем отправим этот код на GitHub или передадим коллеге, то эти данные станут доступны всем, у кого есть доступ к коду. Именно поэтому мы используем `.env` — файл с переменными окружения не попадает в репозиторий, и наши секретные данные остаются в безопасности.

2. После указания модели прописаны параметры генерации изображения:

```
"seed": None
```

`Seed` — это отправная точка для генерации изображения. Все современные нейросети работают не с самими изображениями, а с векторами и сложной математикой. `Seed` — это своего рода начальное состояние, от которого модель начинает строить результат. Если указать конкретное значение `seed`, то при повторном запросе с тем же промптом мы получим очень похожее изображение. Это полезно, когда нужно воспроизвести результат. В нашем случае мы оставляем `seed = None`, то есть позволяем модели генерировать изображение полностью с нуля каждый раз.

3. Следующий параметр — это `aspectRatio`, соотношение сторон изображения (ширина к высоте). Мы можем указать разные форматы:

- `1:1` — квадратное изображение;
- `16:9` — широкоформатный (как у мониторов);
- `9:16` — вертикальный формат (как у телефонов);
- `2:1` — удлинённый горизонтальный формат.

Мы используем `1:1`, потому что работаем с логотипами, которые в основном квадратные.

```
"aspectRatio": {  
  "widthRatio": "1",  
  "heightRatio": "1"
```

При необходимости можно изменить этот параметр позже.
Действия

4. **Messages** — это текстовое описание того, что мы хотим увидеть на изображении. Это список, то есть мы можем передать несколько сообщений с разными параметрами.

```
"messages": [  
  {  
    "weight": "1",  
    "text": prompt  
  }  
]
```

У каждого сообщения есть два основных параметра:

- **text** — сам текст промпта для генерации;
- **weight** — вес этого промпта, то есть насколько сильно нейросеть должна учитывать именно этот текст. Чем больше вес, тем выше приоритет этого текста при генерации.



В следующих уроках мы будем работать с несколькими сообщениями: например, запрос пользователя будет иметь максимальный вес, а предзаготовленные промпты для фона или стиля («минимализм», «неоновые цвета») — меньший вес, но всё равно будут учитываться моделью.

5. **Headers** — это заголовки запроса, которые передаются вместе с телом запроса. Если тело запроса (промпт, параметры) — это то, что мы хотим получить, то заголовки — это информация о том, как мы это передаём и кто мы такие.

```
headers = {  
  "Content-Type": "application/json",  
  "Authorization": f"Api-key <>"
```

Заголовки содержат два обязательных поля:

- **Content-Type** — указывает формат передаваемых данных. В нашем случае это `application/json`, то есть мы отправляем данные в формате JSON;
- **Authorization** — ключ авторизации. Именно через этот заголовок сервис идентифицирует нас и понимает, что запрос пришёл от авторизованного

пользователя.

6. Вместо `<>` в коде нужно прописать наш API-ключ:

```
"Authorization": f"Api-Key {os.getenv('YANDEX_API_KEY')}}"
```

Именно благодаря заголовку авторизации сервис понимает, что это действительно мы, и даёт нам доступ к генерации. Без правильного ключа в заголовках запрос будет отклонён.



Обрати внимание на формат авторизации: здесь мы снова используем `os.getenv()` для получения ключа из `.env`, а перед ключом обязательно указываем префикс `Api-Key` (с пробелом после него).

Префикс авторизации может отличаться в зависимости от платформы. Для Яндекса это `Api-Key`, для некоторых других сервисов — `Bearer`. **Всегда сверяйся с документацией конкретного сервиса**, чтобы использовать правильный формат.

Теперь у нас есть полное представление о структуре запроса: мы указали модель, настроили параметры генерации, передали промпт с нужным весом и добавили заголовки с авторизацией.

7. Здесь предлагаем вернуться назад, к параметру `seed`. Наша функция, как мы уже сказали, принимает два параметра — `prompt` и `seed`.

```
def yandex_art_request(prompt, seed):
```

8. Здесь можно прописать `seed = seed` (вместо `None`):

```
def yandex_art_request(prompt, seed):
    prompt = {
        "modelUri": f"art://{os.getenv('YANDEX_CLOUD_ID')}/yandex-art/late
        "generationOptions": {
            "seed": seed,
```



Действия

Логика здесь простая: какое значение `seed` мы передали в функцию, такое значение и уходит в запрос к серверу.

9. А уже ниже, в тестовой части кода, чуть позже мы в явном виде зададим `seed` :

```
prompt = "Код на мопеде"  
seed = None
```

Это означает, что в текущем варианте мы не фиксируем зерно генерации. Каждый раз изображение будет создаваться заново, с нуля, без попытки воспроизвести предыдущий результат.

Если в дальнейшем нам понадобится управлять повторяемостью — мы просто подставим конкретное значение `seed` в тестовой части кода, а саму функцию при этом менять не придётся.

Таким образом, функция остаётся универсальной, а поведение генерации мы контролируем в момент вызова. Это удобный и правильный подход.

▼ Генерация изображений и сохранение результата

1. Теперь нужно сделать сам запрос. На этом этапе мы впервые реально идём в сервис Яндекса и как бы говорим: «Мне нужно создать изображение по вот этому промпту, я работаю с такого-то аккаунта, начинай генерацию». За это отвечает эта строка кода:

```
create_request = requests.post('https://llm.api.cloud.yandex.net/found  
                                headers=headers, json=prompt)
```

Здесь мы делаем `POST`-запрос к сервису Яндекса по конкретному адресу — это целевой эндпоинт для асинхронной генерации изображений.

В запрос мы передаём:

- `headers` — заголовки запроса, где у нас лежит способ авторизации и API-ключ;
- `json=prompt` — тело запроса, то есть тот самый промпт со всеми параметрами генерации.

Действия

Это и есть тело запроса, которое уходит в Яндекс.

2. После этого получаем ответ.

```
print(done_request.json())
```

3. Итак, этим кодом мы отправили на сервис Яндекса, по сути, запрос «Создай мне такое-то изображение», но генерация не происходит мгновенно. Сервер Яндекса начинает вычисления, и на это требуется время — иногда довольно заметное. Поэтому дальше мы переходим к циклу ожидания:

```
while True:  
    time.sleep(5)
```

Мы делаем паузу в 5 секунд, чтобы не обращаться к серверу слишком часто. Каждые 5 секунд мы будем проверять, завершилась ли генерация.

4. Для этого мы отправляем второй запрос — уже на другой адрес.

```
done_request = requests.get(f'https://llm.api.cloud.yandex.net:443  
                             headers=headers)
```

Когда мы сделали первый запрос, Яндекс вернул нам `id` операции. Этот `id` — уникальный идентификатор конкретной генерации, который мы прописываем во втором запросе. Именно по нему мы узнаём, закончил ли Яндекс генерацию (параметр `done`).

5. Чтобы лучше понимать, что именно возвращает сервер, добавим вывод ответа:

```
print(done_request.json())
```

6. Дальше проверяем статус операции.

```
if done_request.json()['done'] == True:
```

Действия

Ключ `done` означает, завершилась генерация или нет. Если `done == True`, значит, изображение готово, и можно забирать результат.

7. На этом этапе изображение уже пришло в ответе, но оно всё ещё находится в формате `base64` — в закодированном виде. Нам нужно превратить его в нормальный JPEG-файл. Для этого открываем файл на запись в бинарном режиме:

```
with open(create_request.json()['id'] + '.jpeg', 'wb') as file
```

- название файла — это `id` генерации (который мы получили ранее) + `.jpeg`. Так у нас всегда будет уникальное имя файла, соответствующее конкретному запросу.
- режим `wb` означает **write bytes** — запись в байтах. Изображения нельзя корректно записывать как текст, потому что это не человекочитаемые данные, а набор байтов.

8. Теперь используем библиотеку `base64`, чтобы декодировать изображение:

```
file.write(base64.b64decode(done_request.json()['response'
```

Здесь мы:

- берём изображение из ответа;
- декодируем его из `base64`;
- записываем результат в файл.

9. После этого генерация успешно завершена, файл сохранён. Выходим из цикла:

```
break
```

10. Дальше функция просто возвращает имя файла:

```
return create_request.json()['id'] + '.jpeg'
```

Действия

▼ Тестирование

1. Переходим к тестированию. Прописываем промпт: «Кот на мопеде» и `seed` :

```
# тестовые данные
prompt = "Кот на мопеде"
seed = None
```

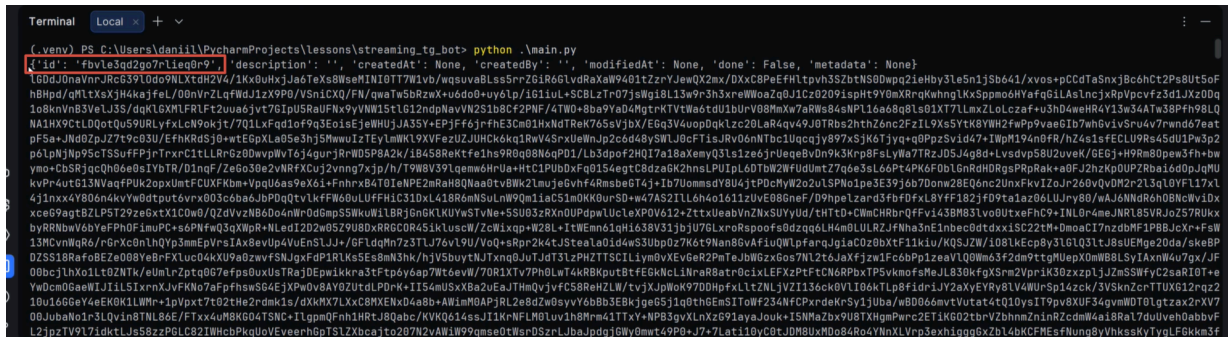
Это максимально простой промпт, поскольку сейчас нам важно не качество изображения, а сам факт, что всё работает.

2. Вызываем функцию:

```
print(yandex_art_request(prompt, seed))
```

3. Запускаем файл в консоли командой `python main.py`. Видим, что запрос ушёл, что нам вернулся `id`. Сервер возвращает изображение в закодированном виде.

Нас в первую очередь интересует именно `id` — это уникальный идентификатор нашей генерации:



```
(.venv) PS C:\Users\daniil\PycharmProjects\lessons\streaming_tg_bot> python .\main.py
{"id": "fbviesqd2go7r1ieq0r9", "description": "", "createdAt": None, "createdBy": "", "modifiedAt": None, "done": False, "metadata": None}
L6ddJ0naVnrJRcG391Ldo9NLxtDh2V4/1Kx0uHxjJa6Texs8WseMINI0T7W1vb/wqsuvabLss5nrZ6IR6LvdRaXaW9401tZzrYJewQX2m/DXxC8PeEfhLtpvhs3ZbtNS00wpq21ehby3le5n1jSb641/xvos+pcCdTaSnxjBc6hCt2Ps8UT5oF
hBpd/gHltXa3jH4kajfeL/00nVr2LqfWdJ1zX9P0/V5n1CXQ/FN/qwaTw5bRzWx+u6do0+uy6lp/1G11uL+SCBLzTr07jsWg18L13w9r3h3kreWwoaZq8J1C28209isphT9Y0mXRqkwhngLkXsppmo6HyafqG1LAslncjxRpPcvfz3d1JXz0Dq
1o8kNv8SveLJ35/dqL6XMLFRfL2uua6jvt761p0SraufXx9yVNM15tL612ndpNavVW251b0Cf2PNF/4TW0+8ba9Ya04MgtKTVtW66td1b0rV08Mxw7aRms84sNPL1a68q8Ls1X71Lmx2L0Lczaf+u3hd4weHRA4Y15w34ATw38Pfh98LQ
MA1W9eL1D0eet050UlyxLcNoksj7701LxFeid0f9p3eL4EjgHMHUJAJ35Y+EPFF6j+fh3Em0dXhNdrTe765y1bX/E9q3Y4u0p0pLz2e2BLAR4qy09J0TAb3zh76nc2F2119X5YK8YWM2fwpPv9aeG1b7ahbviV5u4u7mmd6Faet
pF5a+Jh6D2z779e03U/EfhKMSj0+ntE0pXL05e3hJ5Mmw12TeYlMK19XVFz2UJMK6K6c1Rw46SxUwMwJp3c4d6dy8N1J0fF1s1R0d6nTbclUqecjy897x63K61jyeg-a0Rppz51d47+1Wm194dn0FR/hZ4a1sFECLU9Rs45dU1Pz3p2
p6LjMj9p3e5SuffPj+trxnC1LLRn6z0DwypVt6J4pnrJRw05P8A2k/18458Rkctfe1h9R0q08NqPD1/LB3dp02HQ17a18XemyQ31s1ze6jUaqeBvDn9K3Kp8FslYwA77RzJ0514q8d1Lvsdyp58U20veK/GE6j+H9R80pew3fthbw
ymo+CbS8jqc0h6e0s1YbTR/D1nqf/Z6G03e2VNRFXCJ2vnn97xjp/h/T9W8V391qem6HrUe+HTC1PUBDxFq0154egtC8dz6GK2hnsLPUlP4D1bW28FUDmtZ7q6eSsL6Pt4PKAF0b1GnRdH0RgsPRpRak+a0FJ2h2Kp0UPZBba16d0pJqMl
kvPr4ut613NvaqfPuk20pxUmtFCUXFKbm+VpQ4as9eX61+Fnhrx84T0IeNPE2mRaH8QNa0tyBHK2Lmuj6Gvhf4Rmsb6GT4j+1b7UommsdY8U6jtpDcMyW2o2u15PNo1pe3E39j6b7D0ne28E06nc2UnxFvIZoJr260vQv0M2r213qL0YF17x1
411nxx4Y806n4kVye0tpu6vrx003c6ba6JbP0q0tvlkFFW60uLUFH1C310xL418R6mNSuLnW9Qm1iaC5Im0KK0urSD+e47AS21L16h4o16112UvE086neF/D9hpeLzard3fbfdFxl8YFF182jfd9ta1a206LUJry80/wAJ6NmdR6h08NcwiDx
xceG9agtBZLP527ze0xtX1C0w0/QZdVv2NB604nW0d6mpS5WkuW1BRJG6GKLKUYwSTVne+5S0B3zRXn0UPdpwLUCeXPOV612+2ttUeaabVnZNXSUYUd/tHTtD+CwmCHBrQFFv143BMS3Voa0UtxefHC9+INL0r4meJNR185VRJoZ57RUKX
byRRNBw6bYefPhOfImuPC+66PNfWq3qWpR+NledI2D2W05Z9UBDxRRG0R45ikluscW/ZcWixap+W28L+ItWEmn61qhi638V31j3U76LXrRspooFs0dzq6LH4m0LULRZJfNha3E1nbec0dtdxx1SC22tM+DmoaCI7nzdRNF1PB8JcXr+FsW
13MCvnWq6/rGrXChnLhQYp3mmEpVrsIA8evUp4VuEnSLJ+/GfLdQn7z3TL76vL9U/VoQ+sRpr2KtJdJt3LzPHZTSC1Lym0vEv6eR2Pnt6JbW6xGos7NL2t6JaXfjzW1F66bPp1zeavLQ0M63f2dm9ttgUepX0mMB8LSYIAxnM4u7gx/JF
00bcjLh0L1t0ZNTK/eUmLrZptq067Fps0uxUstRaj0Epr1Kra3tFtp0y6ap7Wt6evW/70R1XtV7Ph0LW4KRBKp0t8tEGKNC1Inra8RatR0c1LFXzPtFtCn6RpbXTP5vkmofaMeJL830KfgK5rm2Vpr1K30zxpzLjJzMS5WfYc2saRI0t+e
Yudcm0GaewJ1L15IrnXJvFKNo7aFpfshwSG4EjXp0v08AY02utdLPDrK+IIS4mUSX8a2uEaJThmQvjvFC58REHLM/tvjXpWok9700HpfXlLzNLjVZ1136ck0VL106kTLp8FIdr1JY2aXyEYRy8LV4WurSp14zck/3VSknZcrtTUXG12rqz2
1b0166eY4eEK8X1LWm+1pVpX7t02THe2rmdk1/dxMKX7LXcC8XENx04a8b+AWIm0APJL2e0dZw0svvY6bB3E8kjge551q0th0EnS1ToWf234NFCPrndKrsYlJuba/wB066mvtUtat4tQ10Ys1T9p8XUF34gvmDDTgtzax2rXV7
0BJuba01r3QvIn8TNL8E/FTxx4uH8K604T5NC+TLpmQFnh1HRTJBQabc/KVWQ614ssJ1IK+NFLH0Uv1h8Hm+41TTxY+NPB3gVnXz891ayaJouk+IGNMz6bx9U8TXHgmPwrc2ET1K602tbrV2bhmZnR2cdm4a18Ra70du0v0ebabbvF
L2jPzTV9171dktLJs58zzPGLC821MhcbKq0QvEveerH6pTS1ZKba3jto207K2vAW1W99mse0tHsr05ZrLJbaJdgJGy0mt49P0+J7+7Lat110Yc0tJDM8UxM084Ro4YmXlVrp3exh1ggg0xZb14bKCFEsfNung8YVhksKyTvgLF6kkn3f
```

Также в этой строке указаны параметры `description` (описание), когда был создан запрос (`createdAt`), кем он был сделан (`createdBy`), завершён он или нет, а также метаданные.

Дальше в выводе идёт огромное количество символов — это и есть содержимое JPEG-файла в base64. Это не человекочитаемый текст, и работать с ним напрямую бессмысленно.

4. Поэтому удаляем вывод закодированного изображения, чтобы консоль не «ломалась»:

```
print(done_request.json())
```

Действия

5. И запускаем скрипт ещё раз. Теперь в консоли всё выглядит аккуратно.

6. Открываем сохранённый в проект файл и видим, что изображение действительно сгенерировано:



7. Запускаем скрипт ещё раз — изображение другое:



Это нормально, потому что промпт очень общий, seed мы поставили, равный None , и никакой воспроизводимости мы не задали.

Итак, изображение генерируется, файл сохраняется, код работает стабильно, и его можно использовать дальше.

8. Возвращаемся в Yandex Cloud и смотрим стоимость. Каждый такой запрос с простым промптом обошёлся в 2.24 рубля.



Попробуйте новый интерфейс объединенного дашборда Yandex Cloud Console



Мы переосмыслили взаимодействие с Console, добавили виджетов и объединили дашборд в единое кастомизируемое пространство

[Перейти к дашборду](#)
[Дашборд каталога](#)
[Сервисные аккаунты](#)
[Уведомления сервисов](#)
[Права доступа](#)
[Операции](#)

	Платежный аккаунт account-412	Баланс 50,00 ₽	Статус Active	Потребление ⓘ 4,47 ₽
--	----------------------------------	-------------------	------------------	-------------------------

Готовые решения
Marketplace

125+ продуктов

0

Подписок >

0

Установленных >

Поиск по сервисам

[Создать ресурс](#)

Списание может происходить не мгновенно — иногда Яндекс показывает потребление с задержкой.

9. Можно перейти в **Сервисные аккаунты** в раздел **Мониторинг**. Здесь видно количество запросов, затраченные ресурсы, токены и операции. Иногда данные появляются с задержкой или не отображаются из-за технических работ, но на работу кода это не влияет.

▼ Итоги урока

Сегодня мы:

- Настроили работу в Yandex Cloud, чтобы иметь возможность отправлять автоматические запросы из кода;
- Разобрали структуру запроса к модели генерации изображений и поняли, какие параметры в нём используются;
- Реализовали запрос к сервису Яндекса для генерации изображения;
- Протестировали работу кода на простом промпте и убедились, что генерация действительно работает.

В следующем уроке мы на основе уже разработанной логики создадим веб-интерфейс на Flask, добавим форму для ввода промпта и отображения результата и сделаем сервис, которым сможет пользоваться любой человек без знания Python.

Дополнительные материалы

▼ Код проекта эксперта

Код проекта эксперта на GitHub
(https://github.com/MrGAN12009/yandex_api)

▼ Ссылки на программы и сайты

<https://console.yandex.cloud> (<https://console.yandex.cloud>) —
консоль Yandex Cloud.

Домашнее задание

1. **Напиши код для веб-приложения**, использующий API **YandexART** для генерации изображений.

Обязательно используй **собственный промпт** (описание изображения), **не тот, который предложил эксперт**.

2. **В качестве домашнего задания пришли скриншоты:**

- написанного кода;
- **сгенерированных изображений**, полученных в результате работы приложения.

На следующем занятии

Мы продолжим разрабатывать веб-приложение для генерации логотипов.

Если у вас есть вопросы, задавайте их, пожалуйста, в учебном чате, а не в этой анкете, — так вы сможете быстрее получить ответ.

Урок

PErr10.1. Генератор логотипов с использованием Yandex-ART API. Часть 1 (

Понравился ли урок?

- ☐ Понравился, все было понятно
- ☒ Неплохо, но не все получилось/не все понятно
- ☐ Не понравился

Обратная связь по уроку (не обязательно)

Отправить

ЗАДАНИЕ

1. **Напиши код для веб-приложения**, использующий API **YandexART** для генерации изображений.

Обязательно используй **собственный промпт** (описание изображения), **не тот, который предложил эксперт**.

2. **В качестве домашнего задания пришли скриншоты:**
 - написанного кода;
 - **сгенерированных изображений**, полученных в результате работы приложения.



Ваш ответ



Действия



максимальный размер файла - 100мб

Прикрепить файлы

Отправить ответ

Сохранить как черновик

20 монет за выполнение этого урока

Следующий урок

Действия